Mastering the game of Go without Human Knowledge

David Silver et al., Nature 2017

2020.12.30

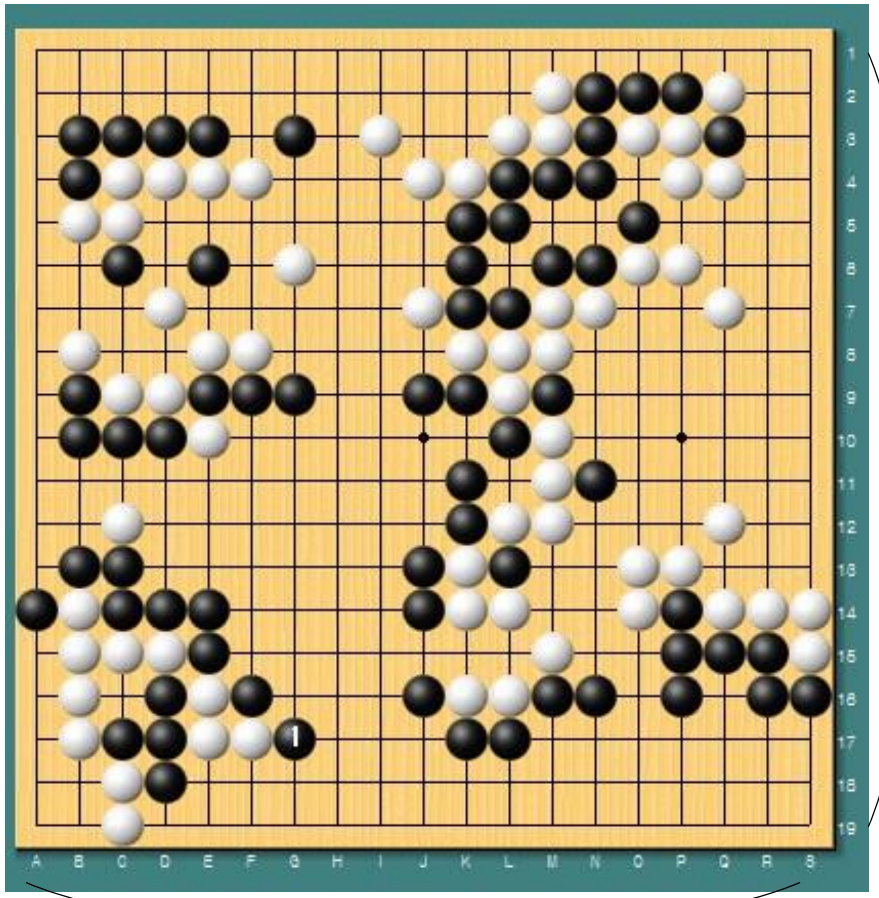
Hojoon Lee

Table of Contents

- Background
  - (1) High-level view for Monte Carlo Tree Search
  - (2) Monte Carlo Tree Search
  - (3) AlphaGo
- Method
  - (1) Training pipeline
  - (2) Network architecture & MCTS
- Results
- Research Trends

Background

Complexity of GO



19

19

Complexity of GO

Around 250 choices, up to 150 turn

$250^{150} \approx 10^{171}$

=20816819938197998469947863334486277028652245388453

05484256394568209274196127380153785256484516985196 4

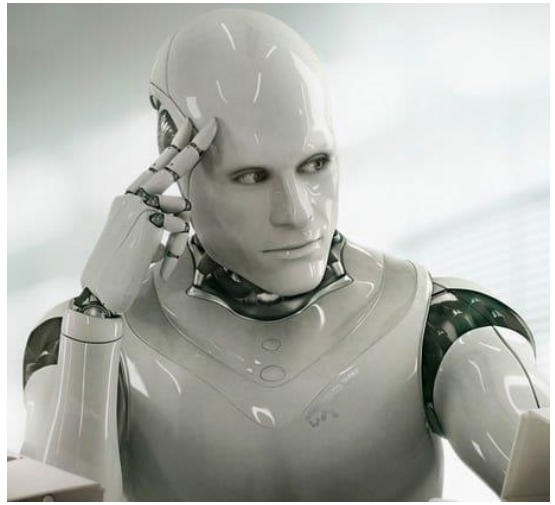
39072599160156281285460898883144271297153193175577 3

6620397247064840935

Impossible for computer to calculate!

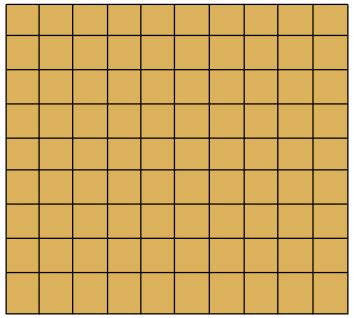
How does the human player plays GO?

What will happen if I place my stone here?

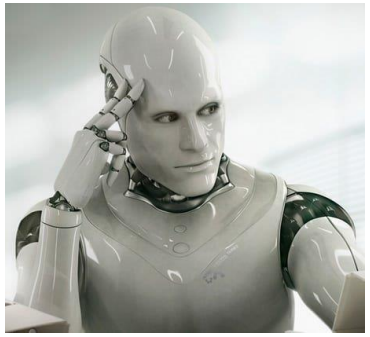


[1] 바둑 삼국지., 박기홍, 랜덤하우스코리아, 2008.

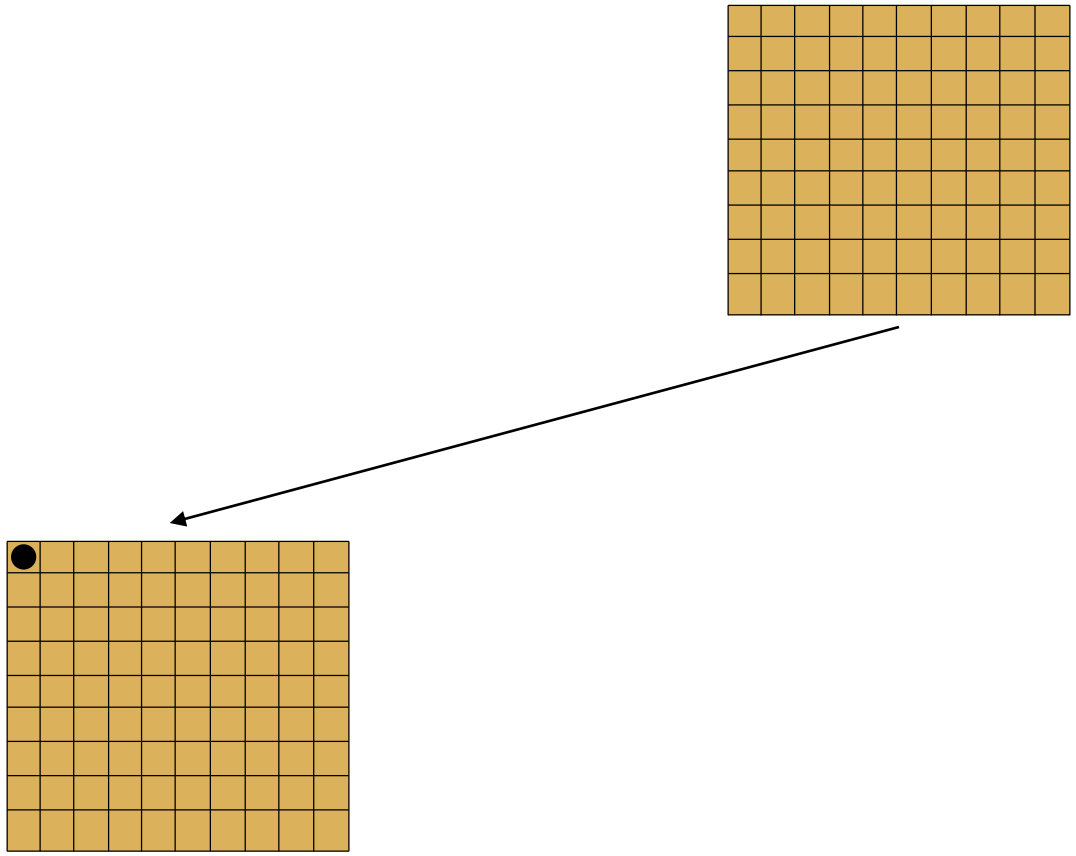
Selection



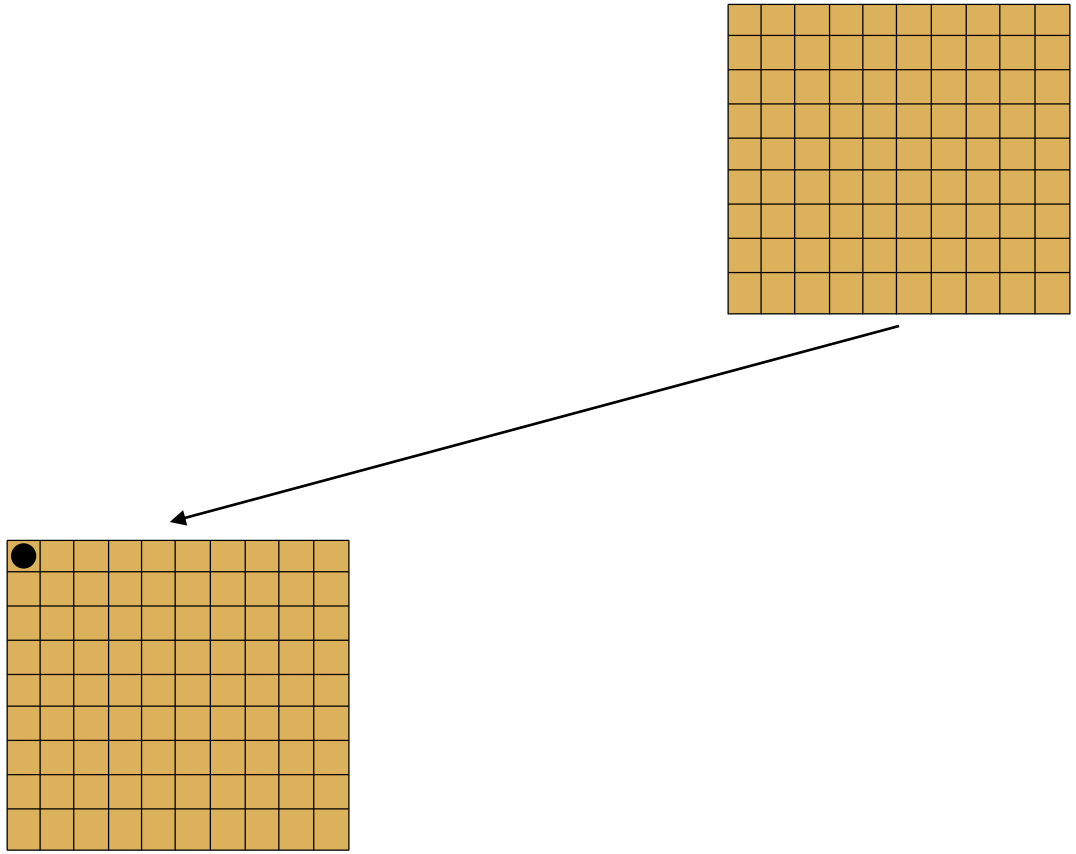
Which action should I do?



Selection

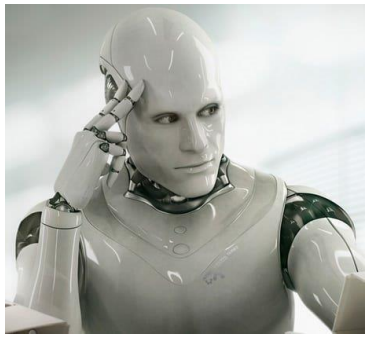


Selection

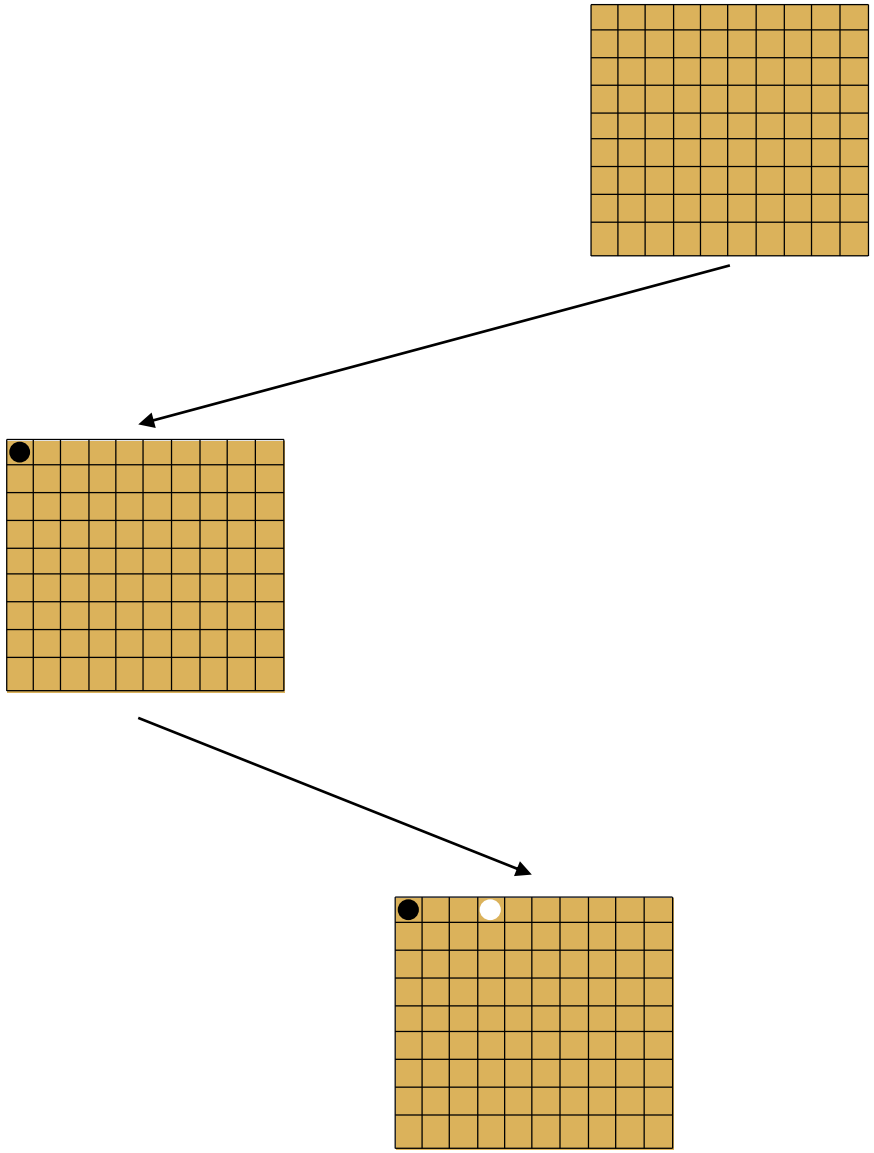


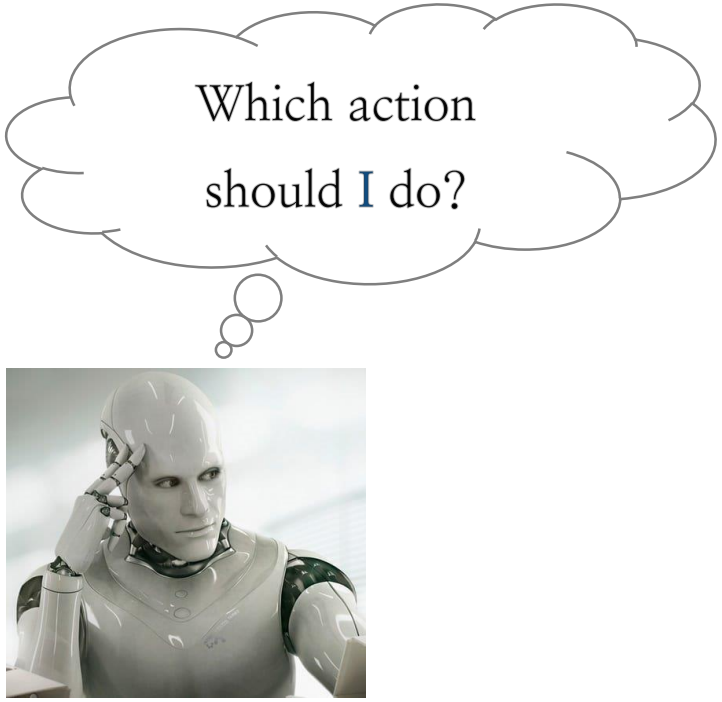
Which action

Opponent may do?



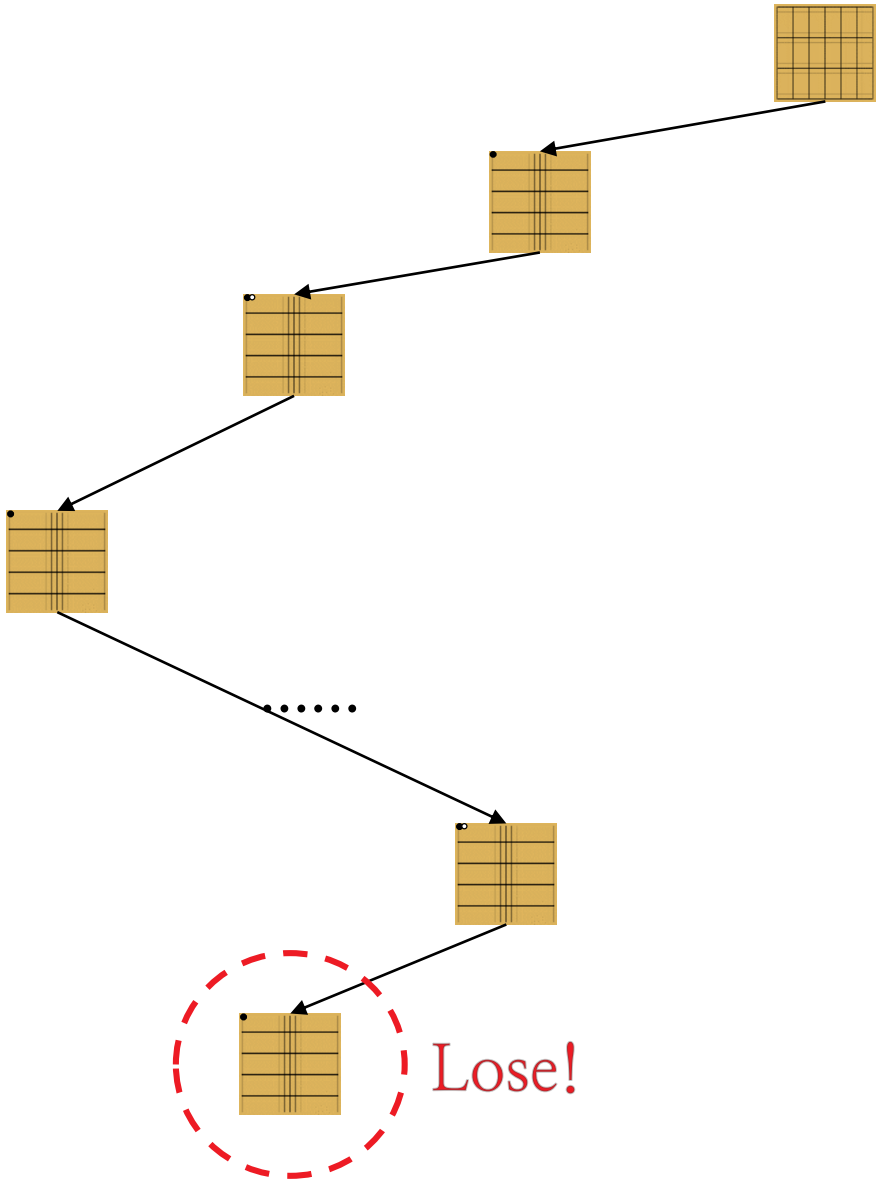
Selection



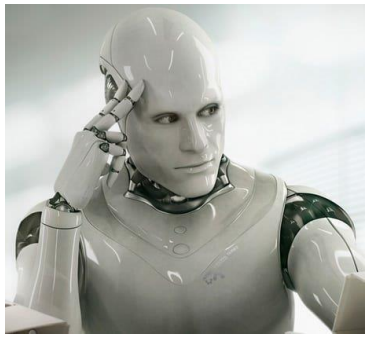


Select the actions until the end of the game

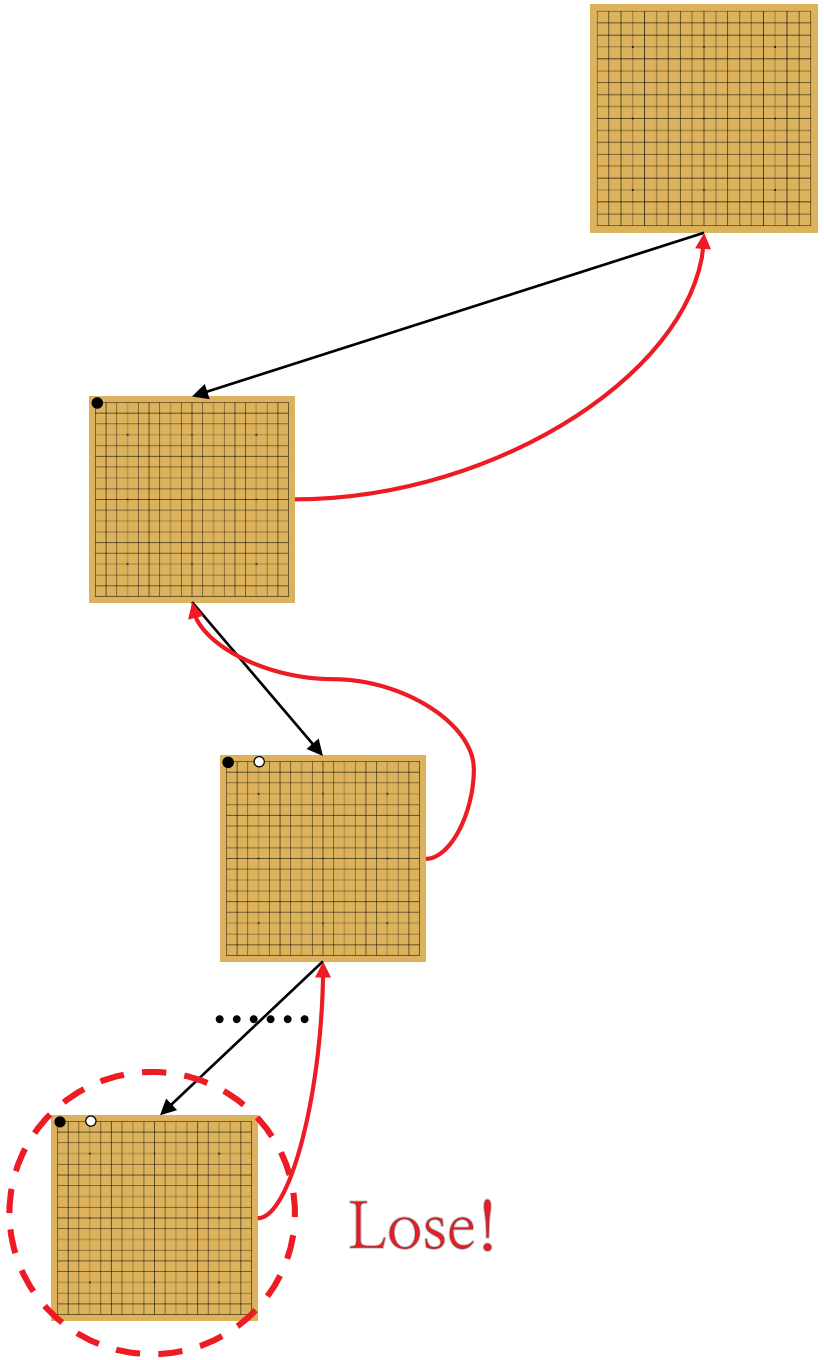
Evaluation



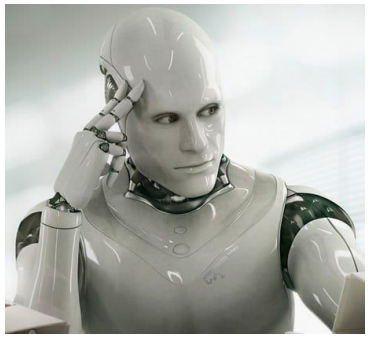
These actions lead to the bad outcome!



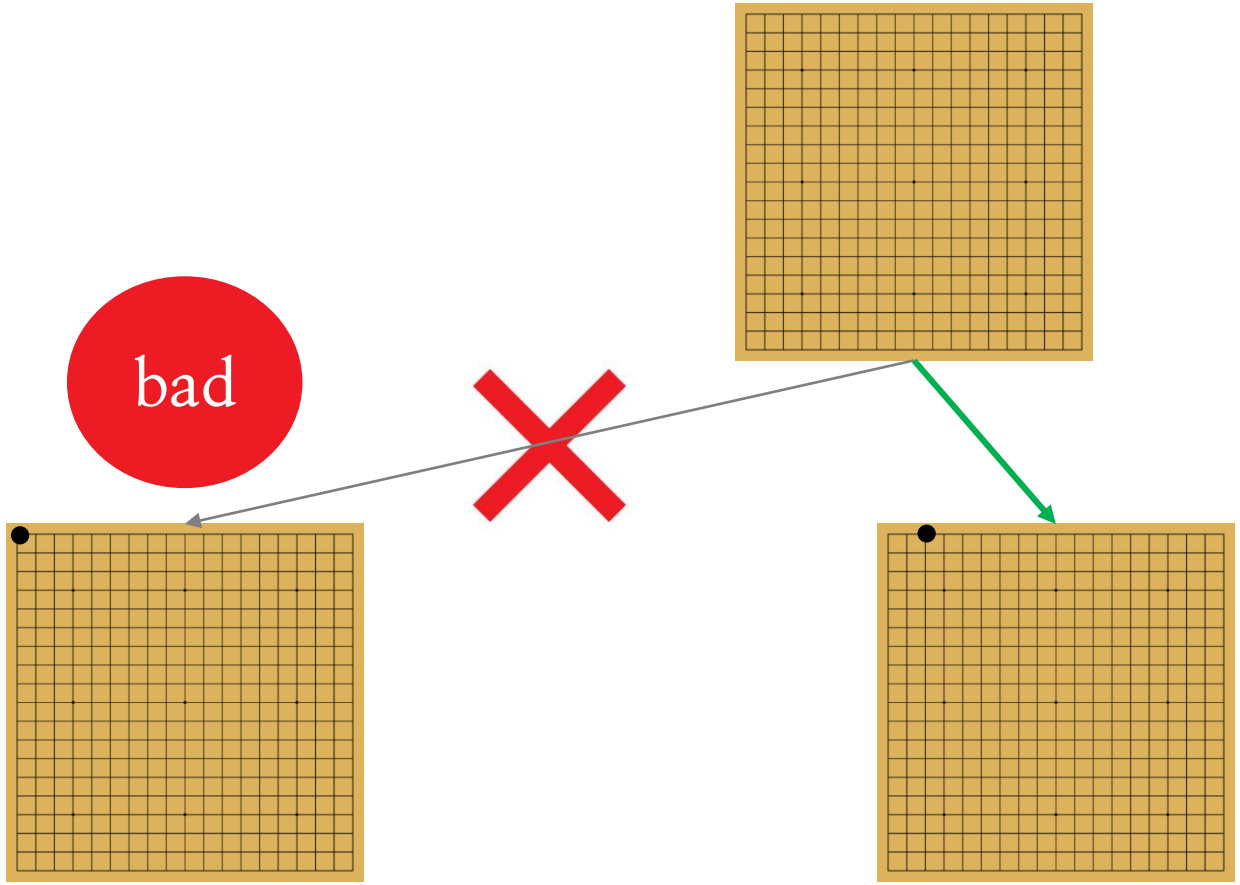
Backup

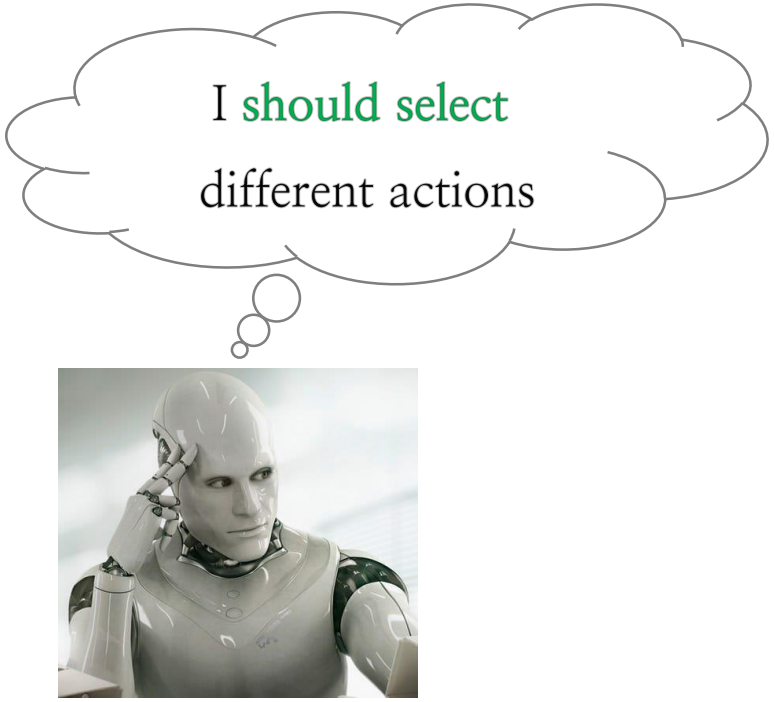


I should not do these actions next time



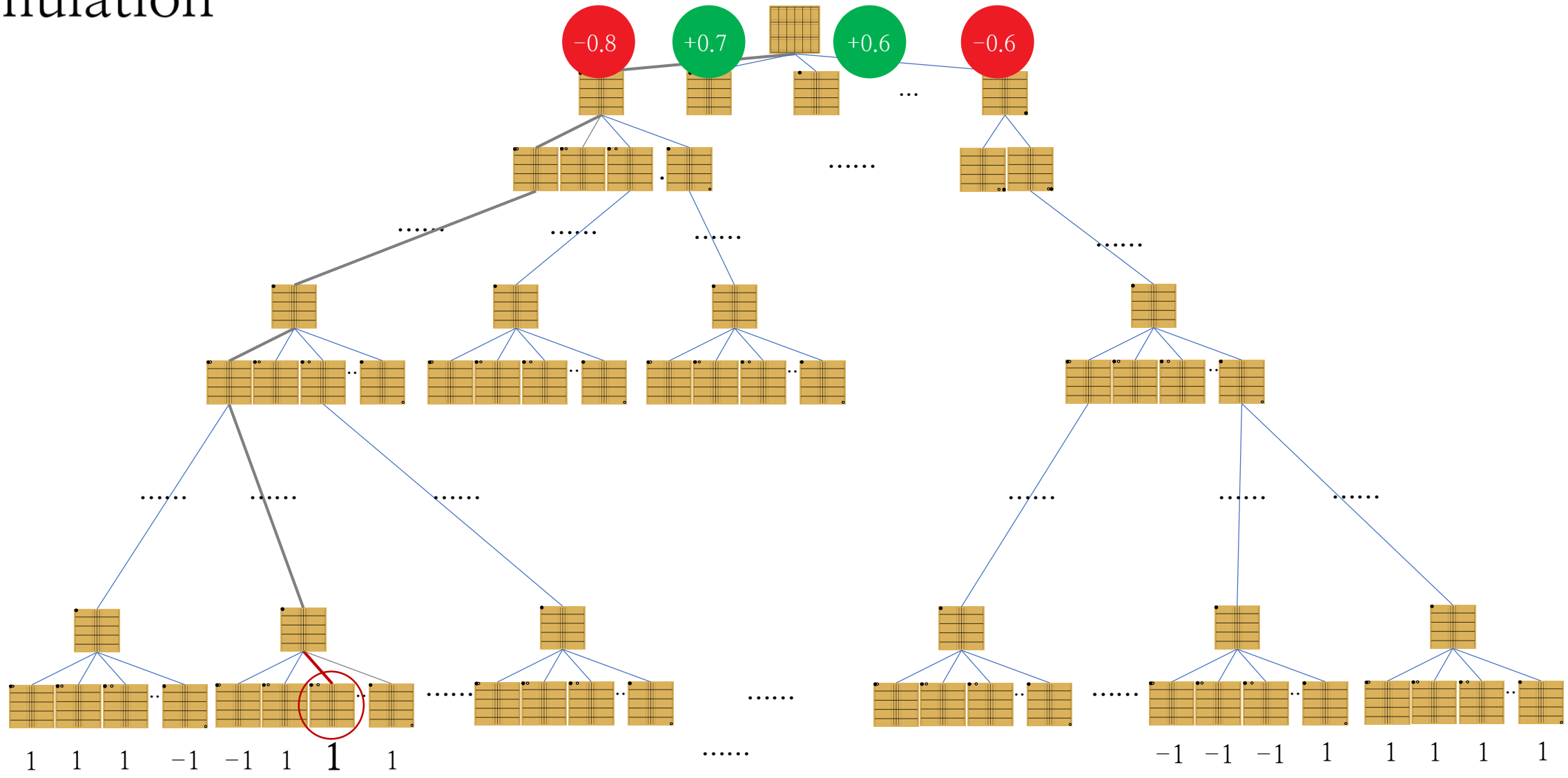
Selection





Repeat this process as much as you can

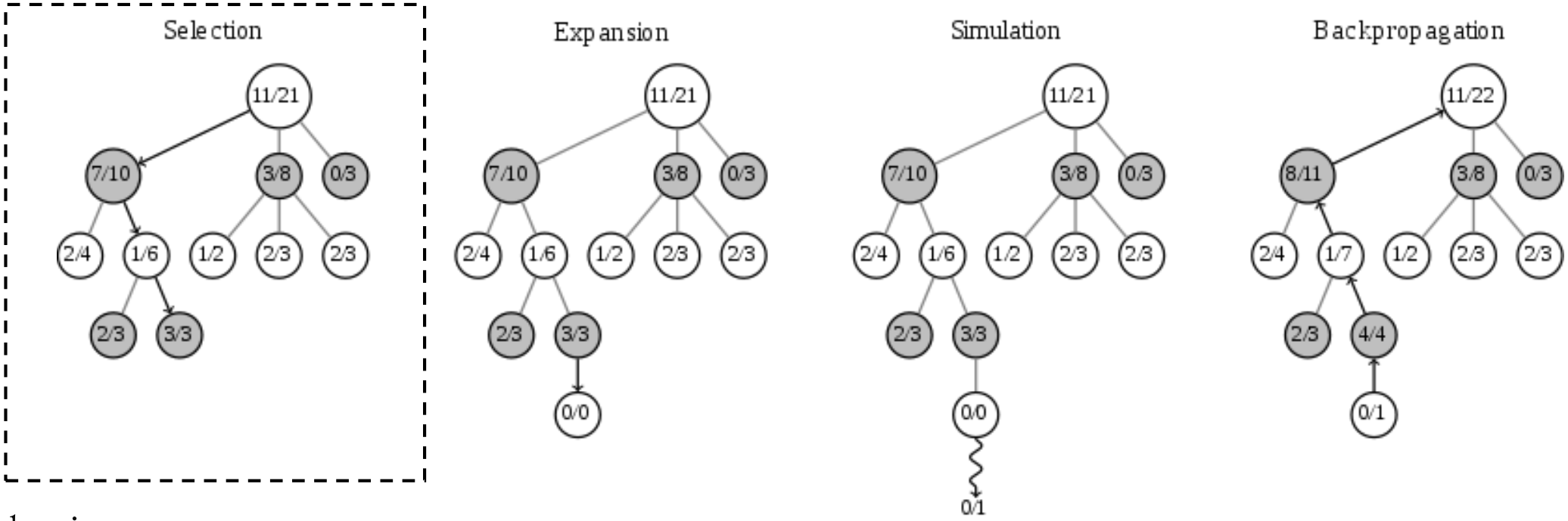
Simulation



The more we repeat, the more we can accurately approximate the score of each action

To more detailed algorithm

Monte Carlo Tree Search



- Selection

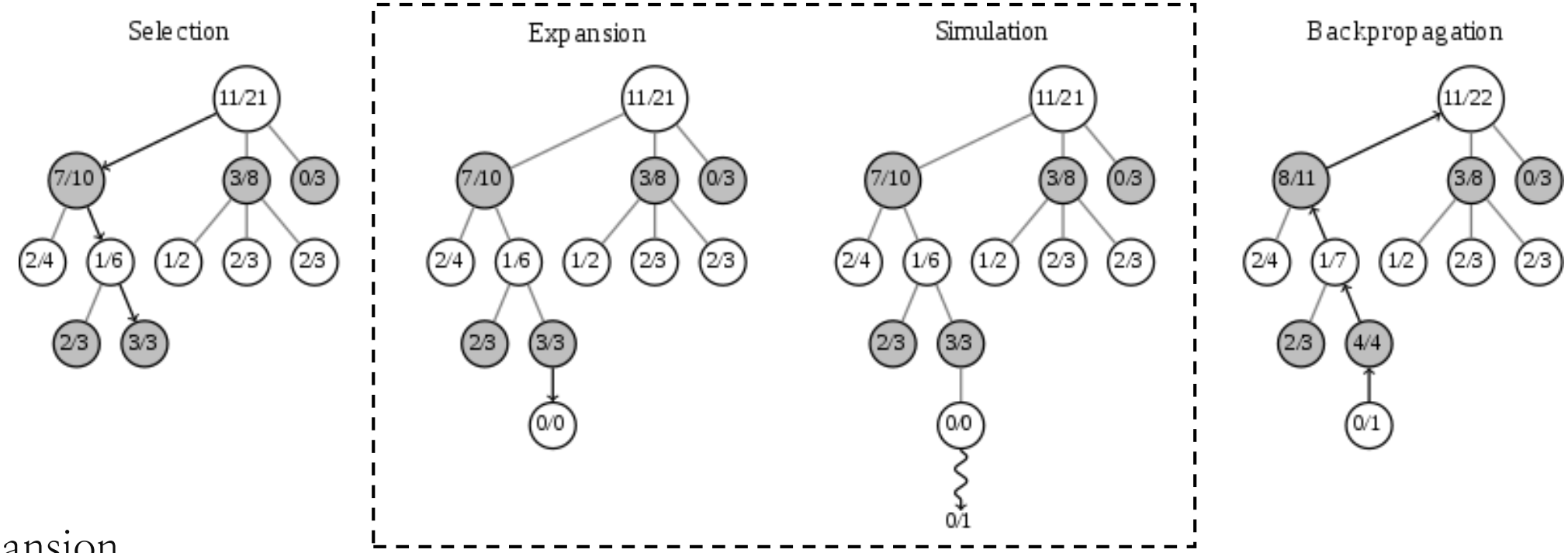
$$a_{select} = \arg\max_{a} Q(s,a) + \frac{\sqrt{N(s)}}{N(s,a)+1}$$

$$Q(s,a) = \frac{W(s,a)}{N(s,a)} : average\ winning\ rate\ of\ (state, action)\ pair$$

$$N(s,a): number\ of\ visit\ counts\ for (state, action)\ pair$$

$$N(s):\ number\ of\ visit\ counts\ for\ state$$

Monte Carlo Tree Search



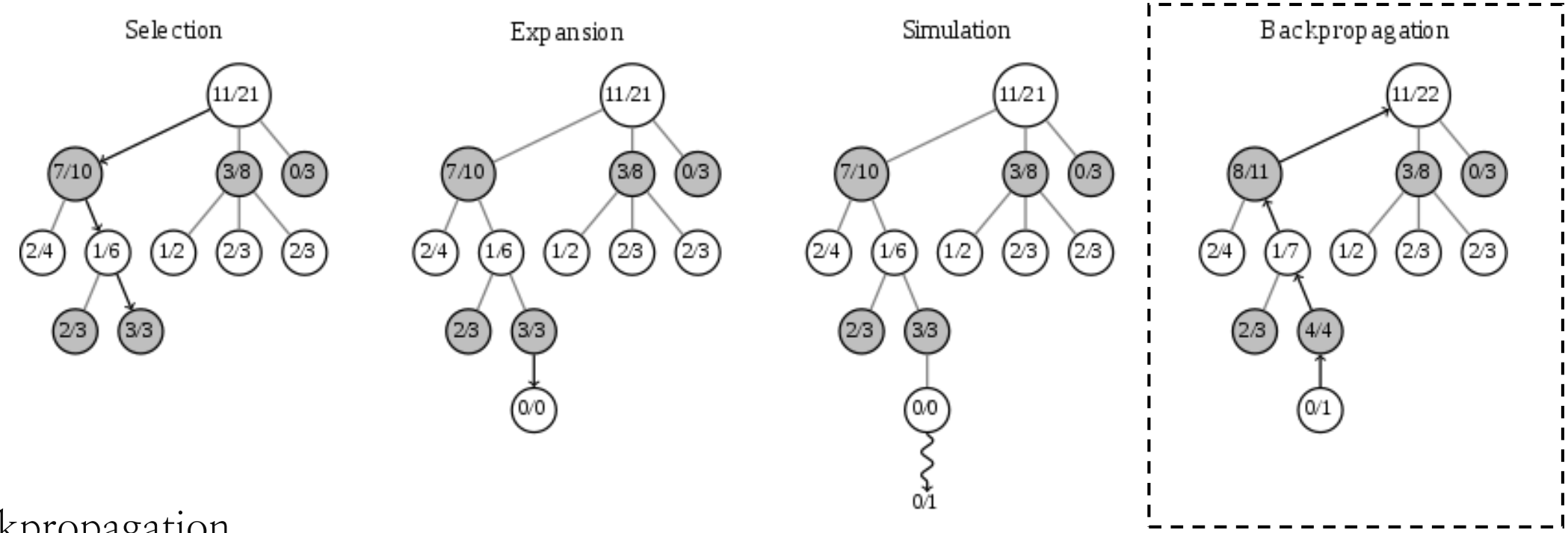
- Expansion

  if the node is visited for the first time, initialized the node statistics

- Simulation

  perform random rollout until the end of the game.
  (any type of the rollout policy could be used)

Monte Carlo Tree Search



- Backpropagation

$if\ the\ node\ is\ visited\ for\ the\ first\ time, initialized\ the\ node\ statistics$

$$N(s,a) = N(s,a) + 1$$

$$W(s,a) = W(s,a) + z\ (outcome\ of\ the\ game)$$

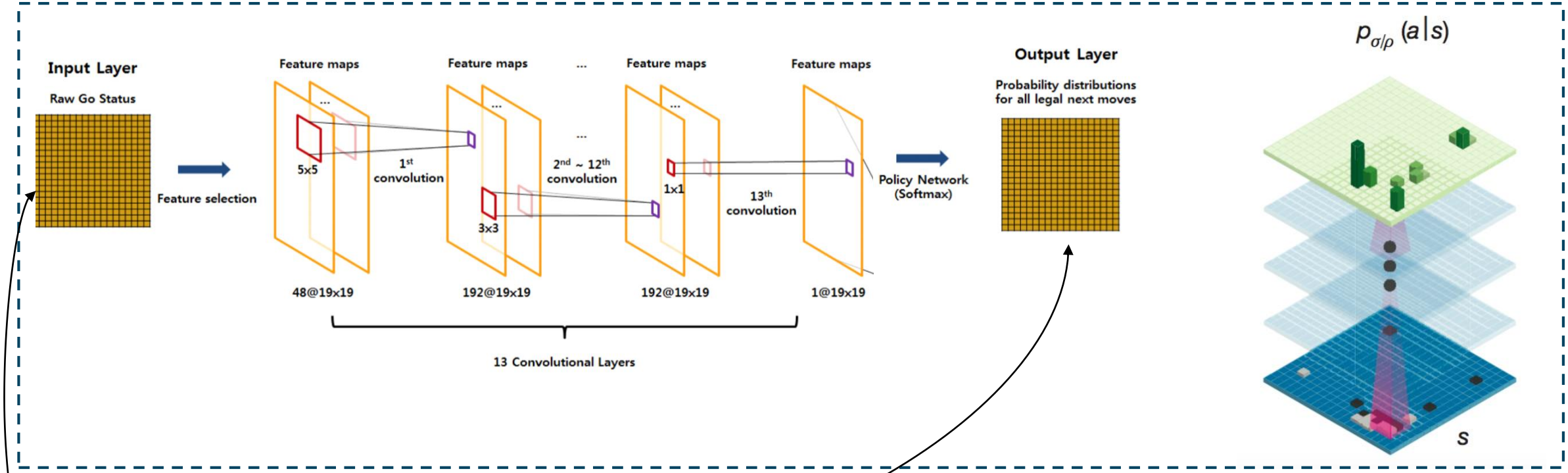
After these simulations, select the most visitied nodes as the next action.
(This selection is for the game-play not for the simulation)

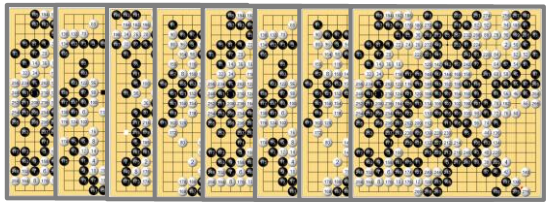
Two problems of Monte Carlo Tree Search

1. **Width** of the Search
2. **Depth** of the Search

→ **AlphaGo** utilized **deep learning** to mitigate these problems

Width of the Search: Policy Network





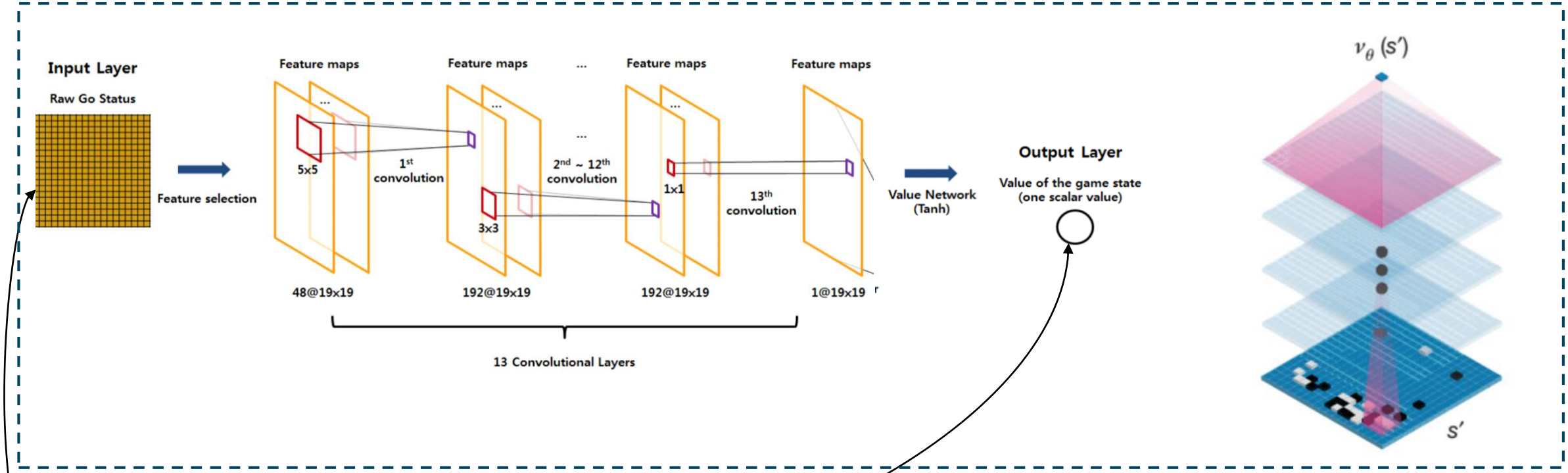
Professional Human Data

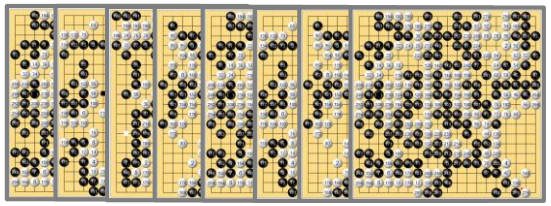
Use Deep Learning to **approximate** the **Promising Moves**!

(*cross entropy loss*)

- Selection: $a_{select} = \arg\max_a Q(s,a) + \boldsymbol{P}(\boldsymbol{a}|\boldsymbol{s}) \frac{\sqrt{N(s)}}{N(s,a)+1}$

Depth of the Search: Value Network





Professional Human Data

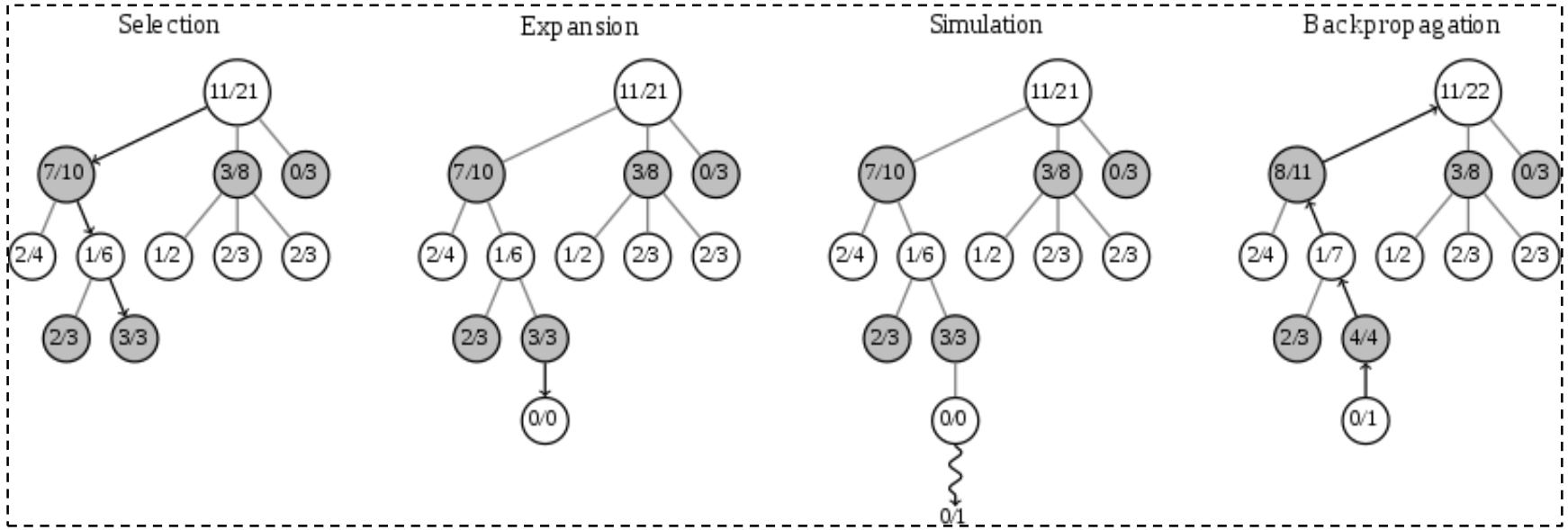
Use Deep Learning to **approximate** the Value of States

$(mse\ loss)$ – Now, we don't need to simulate until the end of the game

- Backup: $W(s,a) = W(s,a) + \boldsymbol{V}(\boldsymbol{s^{next}})$

Summing up the Background

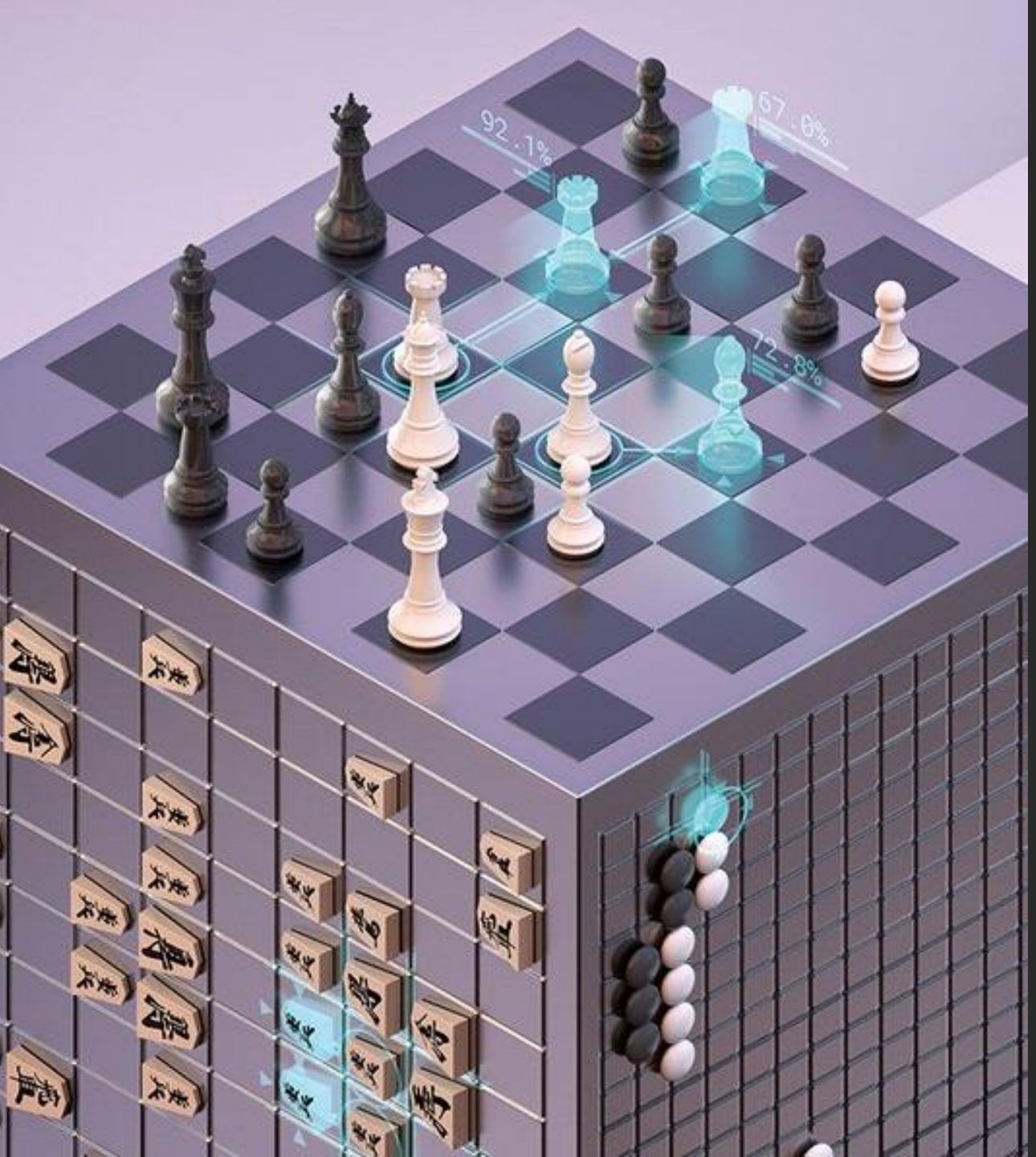
- Monte Carlo Tree Search



Problems: Search space is exhaustive

- AlphaGo

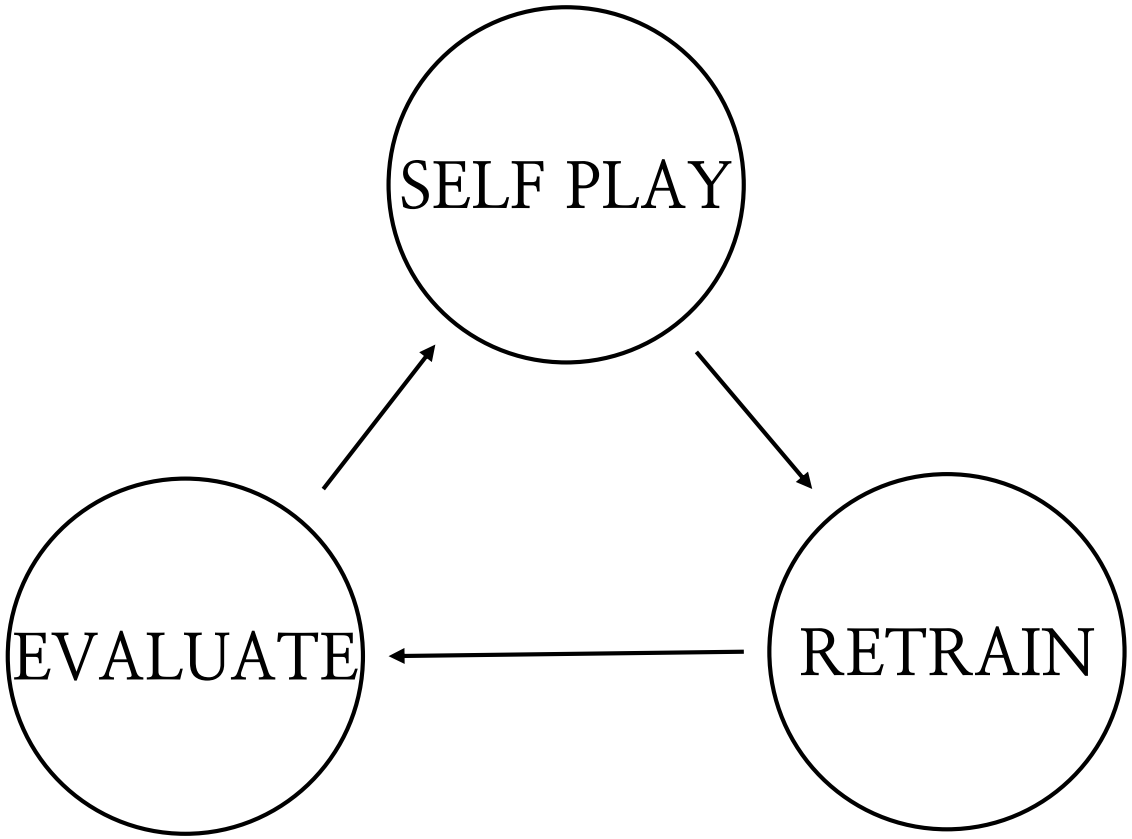
Use deep neural network to reduce the width (policy network) & depth (value network) of the search



However...

Do we even need Human Knowledge?

Training pipeline



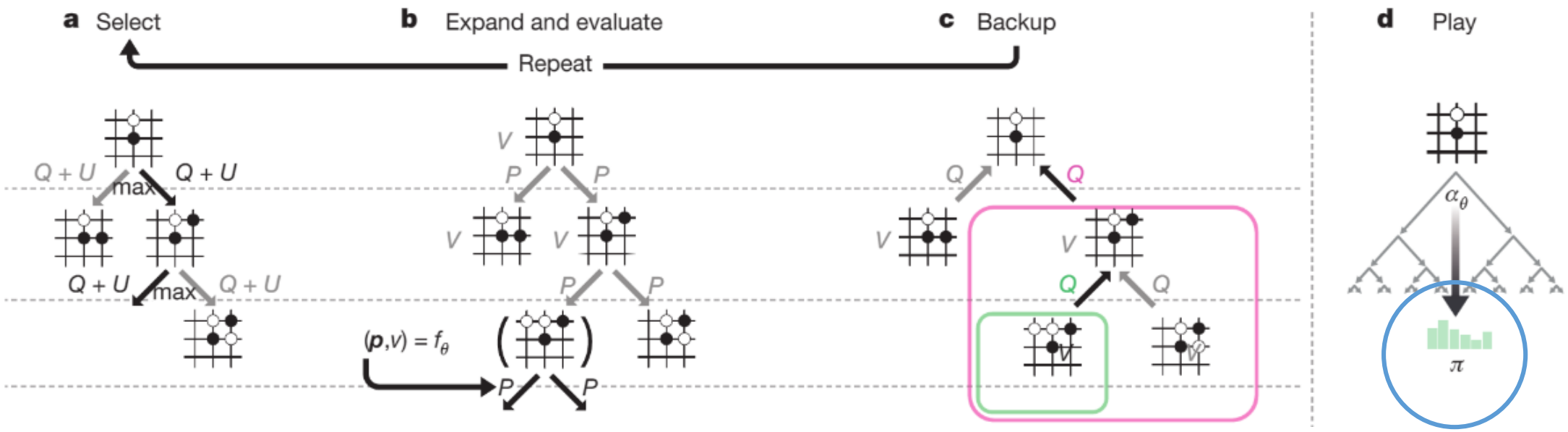
Create the training data from the self-play rather than from the human experts

SELF-PLAY

(1) If network does not exists, randomly initialize the policy network P and the value network V

(2) If network exists, perform self-play with MCTS with the current best P & V

For each move, perform monte carlo tree search



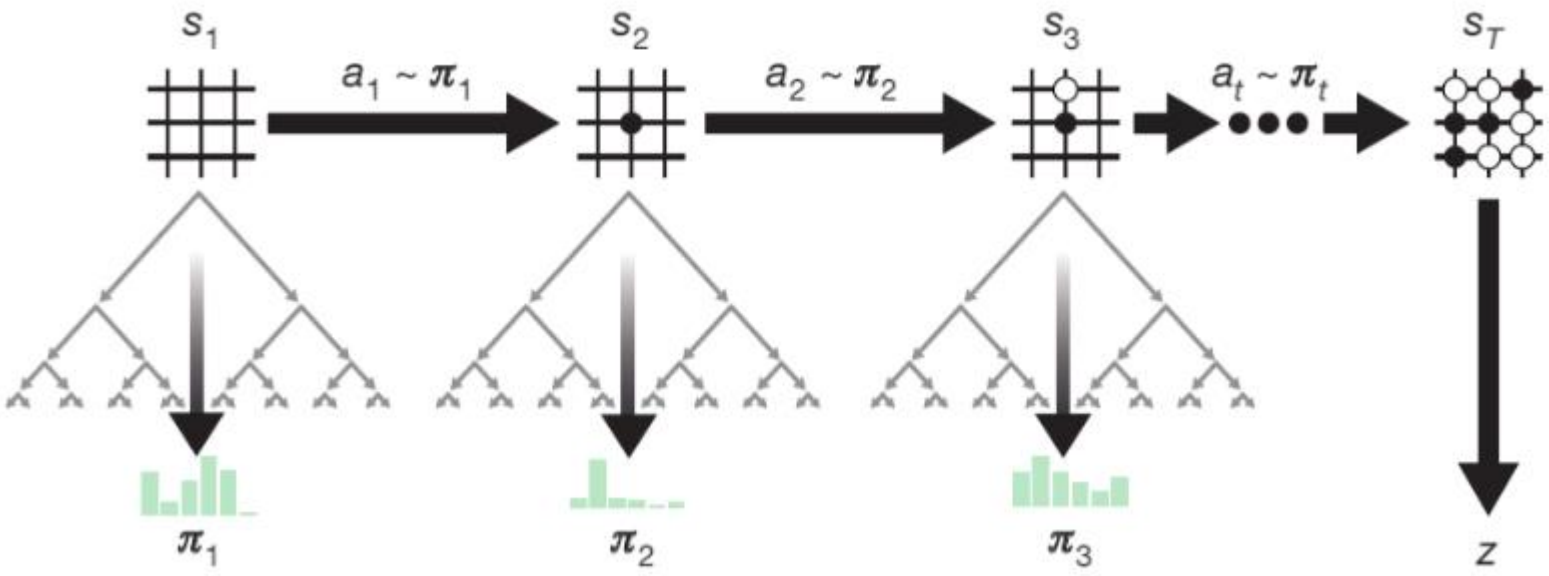
After the simulation, perform action based on the actual search probability by MCTS (**not policy network!**)

SELF-PLAY

(1) If network does not exists, randomly initialize the policy network P and the value network V

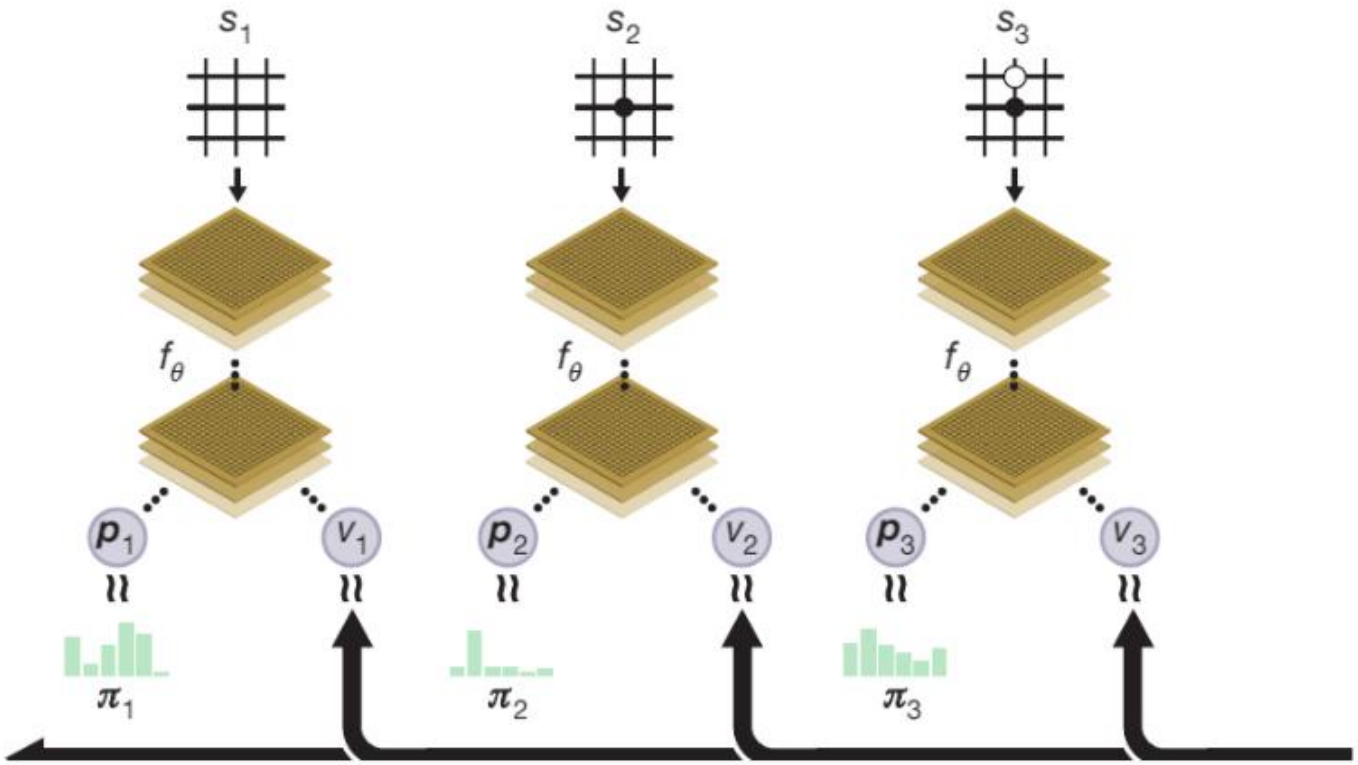
(2) If network exists, perform self-play with MCTS with the current best P & V

(3) For each game, store the z (game outcome) & π (search probability)



RETRAIN

(1) Train the policy network and value network to match the outcome of the self-play



$$(\boldsymbol{p}, v) = f_\theta(s)$$

$$l = (z - v)^2 - \boldsymbol{\pi}^{\mathrm{T}} \log \boldsymbol{p} + c\|\theta\|^2$$

π is the better estimate of the p since it is already bootstrapped by v and z

EVALUATE

(1) Test to see if the new network is stronger

(2) If the new network wins, replace the previous network with the new network

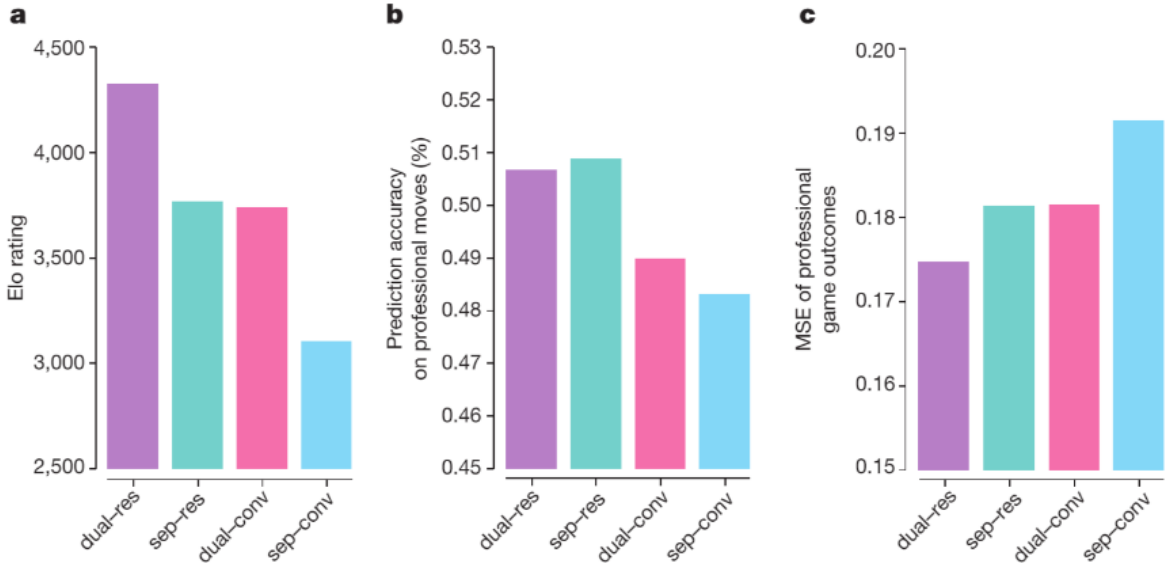
Repeat these process until convergence

Network architecture

Network Input

Neural network architecture. The input to the neural network is a $19 \times 19 \times 17$ image stack comprising 17 binary feature planes. Eight feature planes, X_t, consist of binary values indicating the presence of the current player's stones ($X_t^i = 1$ if intersection i contains a stone of the player's colour at time-step t; 0 if the intersection is empty, contains an opponent stone, or if $t < 0$). A further 8 feature planes, Y_t, represent the corresponding features for the opponent's stones. The final feature plane, C, represents the colour to play, and has a constant value of either 1 if black is to play or 0 if white is to play. These planes are concatenated together to give input features $s_t = [X_t, Y_t, X_{t-1}, Y_{t-1}, \ldots, X_{t-7}, Y_{t-7}, C]$. History features X_t, Y_t are

Multi-head 82 layers residual network



Results

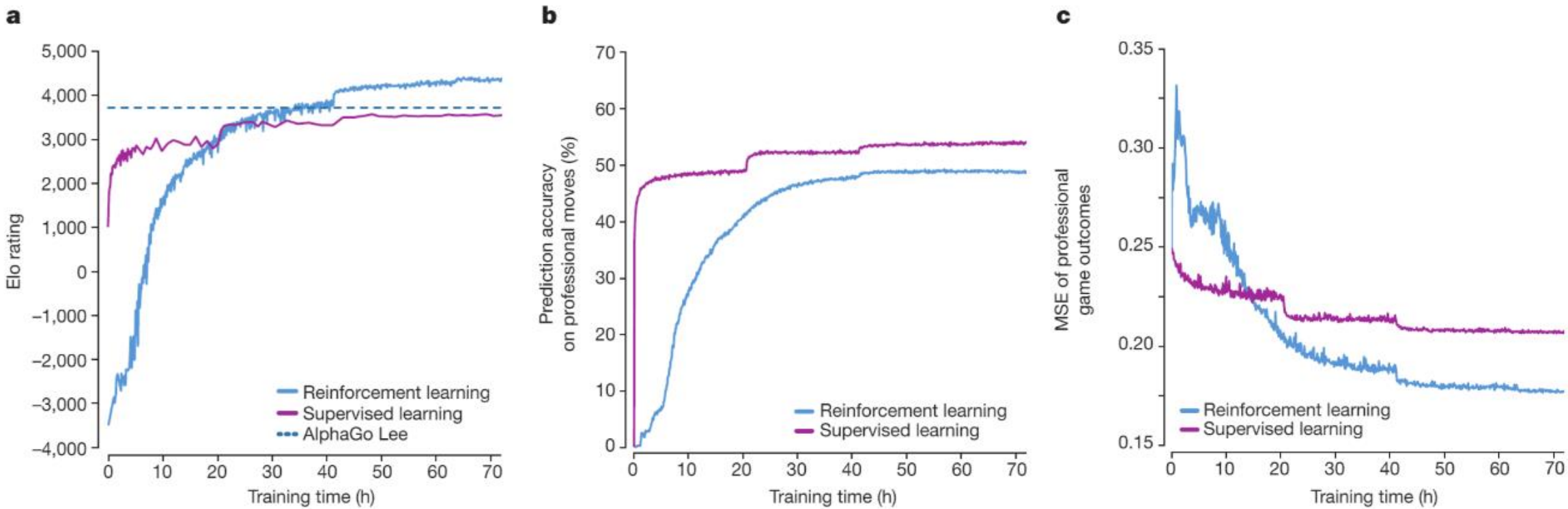


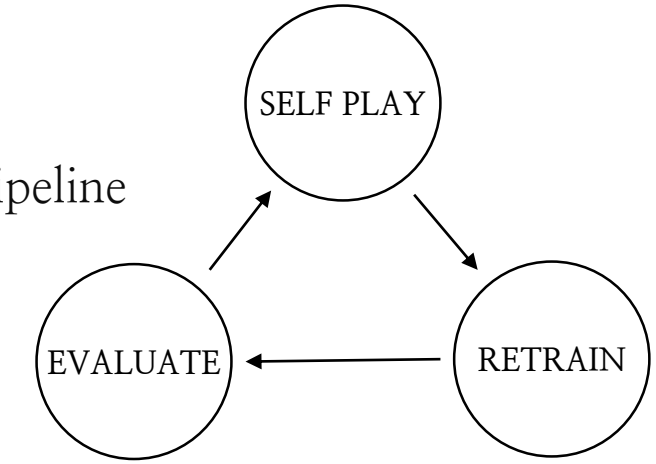
Figure 3 | Empirical evaluation of AlphaGo Zero. a, Performance of self-play reinforcement learning. The plot shows the performance of each MCTS player α_{θ_i} from each iteration i of reinforcement learning in AlphaGo Zero. Elo ratings were computed from evaluation games between different players, using 0.4 s of thinking time per move (see Methods). For comparison, a similar player trained by supervised learning from human data, using the KGS dataset, is also shown. **b**, Prediction accuracy on human professional moves. The plot shows the accuracy of the neural network f_{θ_i}, at each iteration of self-play i, in predicting human professional moves from the GoKifu dataset. The accuracy measures the percentage of positions in which the neural network assigns the highest probability to the human move. The accuracy of a neural network trained by supervised learning is also shown. **c**, Mean-squared error (MSE) of human professional game outcomes. The plot shows the MSE of the neural network f_{θ_i}, at each iteration of self-play i, in predicting the outcome of human professional games from the GoKifu dataset. The MSE is between the actual outcome $z \in \{-1, +1\}$ and the neural network value v, scaled by a factor of $\frac{1}{4}$ to the range of 0–1. The MSE of a neural network trained by supervised learning is also shown.

Summing up

- AlphaGo: MCTS + Supervised Learning
- AlphaZero: MCTS + Reinforcement Learning

  Loss function is still identical to the form of supervised learning

- Training Pipeline



- AlphaZero achieved the state-of-the-art Go performance without any human knowledge

Research Trends

(1) Problems of the MCTS itself

(1) Discrete action space

(2) Not parallelizable (cpu-bounds)

→ Probabilistic Planning with Sequential Monte Carlo Methods., Piche et al., ICLR 2019

: Use Sequential Monte Carlo (=Particle Filtering) methods for simulation.

(2) AlphaZero is only applicable to the perfect information game (know the next state)

→ Combining Deep Reinforcement Learning and Search for Imperfect-Information Games., Brown et al., NeurIPS 2020

: Theoretical extension of AlphaZero methods into Imperfect Information games. (e.g. Poker)

Achieved **State of the art** in the **no-limit Texas hold'em poker!**